

Date _____

Name: Muhammad Farhan

Section: BCS-3k

Roll no: 25k-0538

Course: COAL - Assignment 1.

Course Instructor: Miss Atiya Jakhio

Q.1 (a) Physical Address: 4A5C7h

To find different segment: offset pairs, use use:

$$\text{Offset} = \text{Physical} - (\text{segment} \times 10) \text{ or } 10(\text{hex})$$

Here Physical = 4A5C7h, and we can choose any segment. First, let segment = 4A50

$$\text{Segment} \times 10h = 4A50 \times 10h = 4A500h$$

$$\text{Now offset} = 4A5C7h - 4A500h = 00C7h.$$

$$\therefore \text{Se } \underline{4A50:00C7}$$

$$\begin{array}{r} \text{Proof: } 4A500h \\ + 00C7h \\ \hline 4A5C7h \quad \checkmark \end{array}$$

second, let segment = ~~4A50~~ 48FC

$$\text{segment} \times 10h = \cancel{4A500h} 48FC0h$$

$$\text{offset} = 4A5C7h - \cancel{4A500h} 48FC0h = 1607h.$$

(Note: offset must be between 0000h and FFFFh, otherwise it won't work)

$$\therefore \underline{48FC:1607}$$

$$\begin{array}{r} \text{Proof: } 48FC0h \\ + 1607h \\ \hline 4A5C7h \quad \checkmark \end{array}$$

Proven.

Date _____

Roll no. 25K-0538 , Section: BCS-3k

(b) A logical address is the segment: offset pair (like 4A5C:0007) which is seen by the programmer and assembler. Both are two separate 16-bit values relative to a segment's start. A Physical Address is the actual 20-bit value placed on the address bus, which is one concrete location in the physical memory. It is computed as $\text{physical} = (\text{segment} \times 10h) + \text{offset}$. Because of this formula, many different logical addresses can map to the same physical address (like in a), but the true physical address is different/unique.

The segment: offset pair is combined into a physical address at Execution Time. The assembler only computes offsets relative to a segment's start, it does not know what value will actually be in the segment register at run time. The linker only offsets within or across combined segments and builds relocation info, but still works with relative addresses, not physical.

Only the CPU's Bus Interface Unit (or BIU), at the moment an instruction actually accesses memory, knows the current segment register value and performs the shift and Add to produce the real physical Address.

Date _____

Roll no: 25k-0538

Section: BCS-3k

This has to happen at execution time because segment registers can be reloaded during a program's run (DS pointing to a different data blocks), so the same offset can map to different physical addresses at different points. The binding cannot be fixed any earlier.

Q.2:

AX $\rightarrow$ B , BX $\rightarrow$ E , CX $\rightarrow$ H
DX $\rightarrow$ F , CS $\rightarrow$ C , DS $\rightarrow$ G
SS $\rightarrow$ A , ES $\rightarrow$ D

Q.3: (a) Little-endian: ~~3100~~ 3103h : 00 , 3102h : A3 , 3101h : 4F , 3100h : 00 $\Rightarrow$ 00A34F00h

(b) big-endian: 3100h : 00 , 3101h : 4F , 3102h : A3 , 3103h : 00 $\Rightarrow$ 004FA300h

Date _____

Roll no: 25k-0538 , Section: BCS-3k

(c) If two machines use different endianness, the same sequence of raw bytes will be interpreted as two different numeric values. Even though no data was corrupted in the transfer, the receiving machine must know the sender's byte order (or data converted) to correctly reconstruct the original value. Otherwise multi-byte data will be silently misread.

Q.4: MOV AX, 05F0H AX = 05F0 , CF = 0 , SF = 0
ADD AX, 0A20H AX = 1010 , CF = 0 , SF = 0
MOV BX, 0002H BX = 0002 , CF = 0 , SF = 0
SUB BX, 0005H BX = FFED , CF = 1 , SF = 1
INC AX AX = 1011 , CF = ~~0~~1 , SF = 0
DEC BX BX = FFEC , CF = 1 , SF = 1

1, 1
05F0
+ 0A20

1010

0002
- 0005

FFED

one easy way to calculate Result = (Register max + 1)

increment doesn't affect CF, SF is unchanged

recalculated SF is unchanged

recalculated

(Max = FFFF $\rightarrow$ Max + 1 = 10000h) - (source - destination)

$\therefore$ source = 0005 , destination = 0002 $\rightarrow$ 0005 - 0002 = 0003.

Now 10000h - 0003h $\rightarrow$ FFEDh ✓

Here, we took borrow so CF = 1 , and we have signed number so SF = 1 .

Date _____

Roll no: 25k-0538

Section: BCS-3k

Q.5

Include Irvine32.inc

; Constants below:

Book Price EQU 150

Qty EQU 5

Discount EQU 100

Cash Paid EQU 1000

• Code

main PROC

(a) ; Part (a) below

MOV AX, BookPrice

ADD AX, BookPrice

ADD AX, BookPrice

ADD AX, BookPrice

ADD AX, BookPrice ; new $AX = 150 \times 5 = 750$

(b) ; Part (b) below

SUB AX, Discount ; $AX = \text{Final bill} = 650$

; Part (c) is done at the end.

; part (d) is below/next page

Date _____

Roll no: 25k-0538

Section: BCS-3k

d) MOV BX, CashPaid ~~& RAX~~
SUB BX, AX
; BX = 350 (015E h)

exit

main ENDP

END main

(c) since Final bill is 16 bit, or 2 bytes, meaning it will have 2 blocks of 1 byte memory one after another. Final Bill = 650 = 028A h

in little-endian, the LSB is stored at lower address and MSB at higher.

Address: 5000 h = 8A h

5001 h = 02 h

(e) It would require writing the same code 4 times manually, which is inefficient, long, ^{non-scalable} and prone to mistakes (miscounting). MUL operator fixes it in a single instruction.

Date _____

Roll no: 25k-0538

Section: BCS-3k

Q.6 (a)

.data

~~SCORES~~ SCORES BYTE 6 DUP(0)

(b)

MAXVAL WORD 9999

(c)

NAME BYTE "COMSATS", 0

(d)

BUFFER DWORD 4 DUP(?)

Q.7 1. CountVal EQU 10 → Directive. It defines a constant for the assembler. No machine code is generated.

2. DATA SEGMENT → Directive. Marks start of data segment. No machine code generated.

3. NUM1 DB 25; → Directive. Reserves 1 Byte, no machine code generated.

4. NUM2 DW 100 → Directive. no machine code generated.

5. DATA ENDS → Directive. Marks end of data segment. No machine code.

6. CODE SEGMENT → Directive, no machine code. Marks start of code segment.

Date _____

Roll: 25K-0538

Section: BCS-3k

7. MOV AX, DATA → Instruction: Machine code is generated to load data segment's address into AX.
8. MOV DS, AX → Instruction: Machine code is generated to copy AX into DS register.
9. MOV BL, NUM1 → Instruction: Machine code is generated to load byte at NUM1 into BL.
10. ADD BL, 5 → Instruction: Machine code is generated to add 5 to BL.
11. Code ENDS → Directive: No machine code is generated. Marks end of code segment.
12. END → Directive: No machine code is generated. Tells assembler where the source file ends.

Q.8

I) Total memory to fill = 1mb = 1024 kb.

each chip holds 64 kb, so. $1024 \div 64 = 16$ chips.

To select among among them all, the number of address line needed is given by the rule: $2^{\text{lines}} \geq n$

Here $n=16$, so $2^{\text{lines}} \geq 16 \rightarrow 2^{\text{lines}} \geq 2^4 \rightarrow \text{lines} = 4.$

Date _____

Roll no. 25k-0538

Section: BCS-3k

II) 1. Embedded and resource-constrained systems such as microcontrollers, sensors, appliances, etc. Assembly allows tight control over limited memory and CPU cycles, which compiler generated code can't guarantee.

2. Bootloaders, OS kernels, and device drivers. This code runs before any runtime environment exists and needs to directly manipulate hardware registers and interrupt vectors, which high-level languages can't easily express.

III) Portability: Portability refers to the program's ability to run correctly on different hardware or operating systems with little to no modifications. Assembly is NOT a portable language. Because each chip^s, their architecture family, instruction sets, etc are different from each other. For example: A assembly code written for a x86 process will not work on a chip made ~~for~~ with the ARM architecture without being completely rewritten.

Date _____

Roll no: 25k-0538

Section: BCS-3k

IV) A computer system is built from several layered levels of abstraction. From digital logic $\rightarrow$ Micro Architecture $\rightarrow$ Instruction set Architecture $\rightarrow$ Operating system $\rightarrow$ high level. Each layer behaves like a virtual machine to the layer above it. It hides the complexity of everything beneath it and presents a simpler, self contained interface.

V) A linker combines one or more object files into a single executable program, resolving addresses and references between them along the way. Even a program written entirely in a single file still needs linking because it would typically call methods and routines of external libraries (ex: Irvine library). Linker is what bridges this gap.

VI)

	Real Mode	Protected Mode
Address space	1mb (20-bit, segment $\times$ 10h ^{+offset})	up to 4gb (32-bit addressing)
Memory protection	None.	Processes are isolated
Virtual Memory	Not supported	Supported

Date _____

Roll no: 25k-0538

Section: BCS-3k

VII) This rule exists ^{so that} because the assembler can tell, just from the very first character, whether it's about to read a number or a name. Without it, the assembler would have no reliable way to distinguish between a numeral (like 123) from a name (like 123_{abc}).

VIII) Without the overlap, each instruction would need to be fully fetched before execution could even begin, and the next instruction couldn't start fetching until the current one finished executing. This way, $1+4=5$ cycles would be required for every instruction, done one by one.

With the BIU and EU working in parallel, the BIU can fetch the next instruction (1 cycle) while the EU is still busy executing the current instruction (4 cycles). Since fetching finishes before the execution does, there is no delay and takes 4 cycles instead of five 5.